

Sistema de Controle de Demandas Jurídicas

Especificação Técnica Completa · Versão 1.0

Área de Controladoria · Escritório de Advocacia
Crescer Tecnologia e Gestão · 2025

1. Visão Geral do Sistema

O Sistema de Controle de Demandas é uma plataforma centralizada para gestão operacional e financeira de demandas jurídicas em escritórios de advocacia. Integra identificação de profissionais, núcleos, gestores e prazos em um único ecossistema, eliminando o controle manual por planilhas e protegendo a operação contra perda de prazos processuais.

Problema central	Solução entregue	Resultado esperado
Demandas dispersas em planilhas, sem visibilidade de carga por profissional nem alerta automático de prazo.	Plataforma web com dashboard em tempo real, motor de SLA, RBAC por perfil e integração com TJRS e Power BI.	Zero perda de prazo processual, controladoria com KPIs em tempo real e redução de 60% no tempo de gestão manual.

Arquitetura em 5 camadas

	Portal Web (React SPA) · App Mobile (PWA) · Dashboard Gestão · Painel Admin
	API Gateway (JWT · Rate Limit) · Auth Service (RBAC) · Notification Service
	Demanda Core · Profissional · SLA Engine · Workflow Engine · Audit Trail · KPI Engine
	PostgreSQL (transacional) · Redis (cache/filas) · Object Storage (documentos) · DWH (BI)
	TJRS/TJs · WhatsApp API · Google Drive · Power BI

2. Schema SQL Completo — PostgreSQL

Banco de dados relacional PostgreSQL 15+. Todas as tabelas usam UUID como chave primária, timestamps automáticos e soft-delete via campo ativo. Índices otimizados para as queries de KPI da controladoria.

nucleos — núcleos jurídicos do escritório

```
CREATE TABLE nucleos (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  nome VARCHAR(120) NOT NULL,  
  
  sigla VARCHAR(10) NOT NULL UNIQUE,  
  
  gestor_id UUID REFERENCES profissionais(id),  
  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  
  criado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()  
  
);
```

profissionais — advogados, estagiários e gestores

```
CREATE TABLE profissionais (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  nome VARCHAR(150) NOT NULL,  
  
  oab VARCHAR(30),  
  
  cargo VARCHAR(60) NOT NULL,  
  
  nucleo_id UUID NOT NULL REFERENCES nucleos(id),  
  
  gestor_id UUID REFERENCES profissionais(id),  
  
  carga_maxima INT NOT NULL DEFAULT 30,  
  
  carga_atual INT NOT NULL DEFAULT 0,  
  
  email VARCHAR(200) NOT NULL UNIQUE,  
  
  celular VARCHAR(20),  
  
  -- socio | gestor | advogado | estagiario  
  
  perfil_acesso VARCHAR(20) NOT NULL,  
  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  
  criado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()  
  
);
```

demandas — entidade central do sistema

```
CREATE TABLE demandas (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  titulo VARCHAR(250) NOT NULL,  
  
  descricao TEXT,  
  
  -- trabalhista | tributario | civil | empresarial | ...  
  
  tipo_demanda VARCHAR(50) NOT NULL,  
  
  -- triagem | alocao | execucao | encerrado | arquivado  
  
  status VARCHAR(30) NOT NULL DEFAULT 'triagem',  
  
  -- urgente | alta | normal | baixa  
  
  prioridade VARCHAR(20) NOT NULL DEFAULT 'normal',  
  
  nucleo_id UUID NOT NULL REFERENCES nucleos(id),  
  
  criado_por UUID NOT NULL REFERENCES profissionais(id),  
  
  valor_causa NUMERIC(15,2),  
  
  prazo_legal DATE,  
  
  prazo_interno DATE,  
  
  numero_processo VARCHAR(50),  
  
  tribunal VARCHAR(100),  
  
  fase_processual VARCHAR(60),  
  
  criado_em TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()  
  
);  
  
-- Trigger: atualiza atualizado_em em todo UPDATE  
  
CREATE TRIGGER trg_demandas_updated
```

```
BEFORE UPDATE ON demandas
```

```
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

alocacoes — vínculo profissional <> demanda

```
CREATE TABLE alocacoes (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  demanda_id UUID NOT NULL REFERENCES demandas(id),  
  
  profissional_id UUID NOT NULL REFERENCES profissionais(id),  
  
  -- responsavel | colaborador | revisor  
  
  papel VARCHAR(30) NOT NULL DEFAULT 'colaborador',  
  
  responsavel_principal BOOLEAN NOT NULL DEFAULT FALSE,  
  
  horas_estimadas INT,  
  
  horas_realizadas INT NOT NULL DEFAULT 0,  
  
  alocado_em TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  encerrado_em TIMESTAMPTZ,  
  
  -- ativo | encerrado | transferido  
  
  status VARCHAR(20) NOT NULL DEFAULT 'ativo',  
  
  UNIQUE (demanda_id, profissional_id)  
  
);  
  
-- Índice para queries de produtividade  
  
CREATE INDEX idx_aloc_profissional ON alocacoes(profissional_id);  
  
CREATE INDEX idx_aloc_demanda ON alocacoes(demanda_id);
```

eventos_sla — gatilhos de prazo automatizados

```
CREATE TABLE eventos_sla (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  demanda_id UUID NOT NULL REFERENCES demandas(id),  
  
  -- alerta_72h | alerta_48h | alerta_24h | vencido  
  
  tipo_alerta VARCHAR(30) NOT NULL,  
  
  gatilho_em TIMESTAMPTZ NOT NULL,  
  
  disparado BOOLEAN NOT NULL DEFAULT FALSE,  
  
  -- email | whatsapp | push | todos  
  
  canal VARCHAR(30) NOT NULL DEFAULT 'todos',  
  
  criado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()  
  
);  
  
CREATE INDEX idx_sla_gatilho ON eventos_sla(gatilho_em)  
  
WHERE disparado = FALSE;
```

movimentacoes — audit trail de status

```
CREATE TABLE movimentacoes (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  demanda_id UUID NOT NULL REFERENCES demandas(id),  
  
  profissional_id UUID NOT NULL REFERENCES profissionais(id),  
  
  status_anterior VARCHAR(30),  
  
  status_novo VARCHAR(30) NOT NULL,  
  
  observacao TEXT,  
  
  ocorrido_em TIMESTAMPTZ NOT NULL DEFAULT NOW()  
  
);  
  
CREATE INDEX idx_mov_demanda ON movimentacoes(demanda_id);
```

documentos — peças e arquivos vinculados

```
CREATE TABLE documentos (  
  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  demanda_id UUID NOT NULL REFERENCES demandas(id),  
  
  profissional_id UUID NOT NULL REFERENCES profissionais(id),  
  
  nome_arquivo VARCHAR(300) NOT NULL,  
  
  -- peticao | contrato | procuracao | comprovante | outro  
  
  tipo_documento VARCHAR(50),  
  
  storage_path TEXT NOT NULL,  
  
  tamanho_bytes BIGINT,  
  
  enviado_em TIMESTAMPTZ NOT NULL DEFAULT NOW()  
  
);
```

Função utilitária — set_updated_at()

```
CREATE OR REPLACE FUNCTION set_updated_at()  
  
RETURNS TRIGGER AS $$  
  
BEGIN  
  
NEW.atualizado_em = NOW();  
  
RETURN NEW;  
  
END;  
  
$$ LANGUAGE plpgsql;
```

3. Modelo de Controle de Acesso — RBAC

Quatro perfis hierárquicos controlam o acesso a cada recurso do sistema. A restrição 'próprias' significa que o profissional só enxerga e edita registros nos quais está alocado como responsável principal. Implementado via middleware JWT + verificação de `perfil_acesso` e `nucleo_id` na camada de serviço.

Recurso / Ação	Sócio	Gestor	Advogado	Estagiário
DEMANDAS				
Criar demanda	sim	sim	sim	—
Ver demandas núcleo	sim	sim	próprias	próprias
Ver todas demandas	sim	leitura	—	—
Editar demanda	sim	sim	próprias	—
Encerrar / arquivar	sim	sim	—	—
ALOCAÇÕES E EQUIPE				
Alocar profissional	sim	sim	—	—
Ver carga individual	sim	sim	próprias	próprias
Gerenciar profissionais	sim	leitura	—	—
DOCUMENTOS				
Anexar documento	sim	sim	próprias	próprias
Excluir documento	sim	sim	—	—
CONTROLADORIA				
Dashboard geral	sim	leitura	—	—
KPIs por profissional	sim	sim	próprias	—
Relatórios financeiros	sim	—	—	—
Exportar dados (BI)	sim	leitura	—	—
CONFIGURAÇÃO				
Criar / editar núcleos	sim	—	—	—
Configurar SLAs	sim	leitura	—	—
Gerenciar usuários	sim	—	—	—
Audit trail (logs)	sim	leitura	—	—
sim = acesso total		leitura = somente visualizar		próprias = somente seus registros
				— = sem acesso

Implementação — middleware Python (FastAPI)

```
from functools import wraps

from fastapi import HTTPException, Depends


PERFIL_HIERARQUIA = {

'socio': 4, 'gestor': 3, 'advogado': 2, 'estagiario': 1

}


def require_perfil(minimo: str):

def decorator(func):

@wraps(func)

async def wrapper(*args, usuario=Depends(get_usuario_atual), **kw):

nivel_user = PERFIL_HIERARQUIA.get(usuario.perfil_acesso, 0)

nivel_min = PERFIL_HIERARQUIA.get(minimo, 99)

if nivel_user < nivel_min:

raise HTTPException(403, 'Acesso negado')

return await func(*args, usuario=usuario, **kw)

return wrapper

return decorator


# Uso nos endpoints:

# @router.delete('/demandas/{id}')

# @require_perfil('gestor')

# async def encerrar_demanda(id: UUID, usuario=Depends(get_usuario_atual)):

# ...
```

4. Queries de KPI — Controladoria

Seis queries prontas para uso direto no PostgreSQL ou como datasets do Power BI. Todas usam CTEs para legibilidade e índices já criados no schema para performance. Recomenda-se criar views materializadas para as queries de painel, com refresh a cada 15 minutos via `pg_cron`.

KPI 1 — Produção por Profissional

Demandas ativas, encerradas no mês, horas realizadas e índice de ocupação (%) por profissional e núcleo. Sinaliza sobrecarga antes de virar problema operacional.

```
WITH base AS (  
  
  SELECT p.id, p.nome, p.cargo, n.nome AS nucleo,  
  
  p.carga_maxima,  
  
  COUNT(DISTINCT a.demanda_id)  
  
  FILTER (WHERE d.status NOT IN ('encerrado','arquivado'))  
  
  AS demandas_ativas,  
  
  COUNT(DISTINCT a.demanda_id)  
  
  FILTER (WHERE d.status = 'encerrado'  
  
  AND a.encerrado_em >= DATE_TRUNC('month', CURRENT_DATE))  
  
  AS encerradas_mes,  
  
  SUM(a.horas_realizadas)  
  
  FILTER (WHERE a.encerrado_em >=  
  
  DATE_TRUNC('month', CURRENT_DATE)) AS horas_mes  
  
  FROM profissionais p  
  
  JOIN nucleos n ON n.id = p.nucleo_id  
  
  LEFT JOIN alocaoes a ON a.profissional_id = p.id  
  
  AND a.responsavel_principal = TRUE  
  
  LEFT JOIN demandas d ON d.id = a.demanda_id  
  
  WHERE p.ativo = TRUE  
  
  GROUP BY p.id, p.nome, p.cargo, n.nome, p.carga_maxima  
  
  )  
  
  SELECT *,
```

```
ROUND(demandas_ativas::numeric /  
  
NULLIF(carga_maxima,0) * 100, 1) AS pct_ocupacao,  
  
CASE WHEN demandas_ativas >= carga_maxima THEN 'sobrecarga'  
  
WHEN demandas_ativas >= carga_maxima*.8 THEN 'alerta'  
  
ELSE 'regular' END AS status_carga  
  
FROM base  
  
ORDER BY nucleo, demandas_ativas DESC;
```

KPI 2 — Aderência ao SLA

Percentual de demandas dentro do prazo interno por profissional e núcleo. Demandas vencidas e vencendo em 48h ficam destacadas. Qualquer núcleo abaixo de 85% requer intervenção imediata.

```
SELECT n.nome AS nucleo, p.nome AS profissional,

COUNT(*) AS total_demandas,

COUNT(*) FILTER (WHERE d.prazo_interno >= CURRENT_DATE

AND d.status NOT IN ('encerrado','arquivado'))

AS dentro_prazo,

COUNT(*) FILTER (WHERE d.prazo_interno < CURRENT_DATE

AND d.status NOT IN ('encerrado','arquivado'))

AS vencidas,

COUNT(*) FILTER (WHERE d.prazo_interno

BETWEEN CURRENT_DATE AND CURRENT_DATE + 2

AND d.status NOT IN ('encerrado','arquivado'))

AS vencendo_48h,

ROUND(COUNT(*) FILTER (

WHERE d.prazo_interno >= CURRENT_DATE

OR d.status = 'encerrado')::numeric

/ NULLIF(COUNT(*),0) * 100, 1) AS pct_aderencia_sla

FROM demandas d

JOIN alocaoes a ON a.demanda_id = d.id

AND a.responsavel_principal = TRUE

JOIN profissionais p ON p.id = a.profissional_id

JOIN nucleos n ON n.id = d.nucleo_id

GROUP BY n.nome, p.nome
```

```
HAVING COUNT(*) > 0
```

```
ORDER BY pct_aderencia_sla ASC; -- piores primeiro
```

KPI 3 — Aging de Demandas

Estoque de demandas em aberto classificadas por faixa de tempo (normal / monitorar / atenção / crítico). Demandas críticas acima de 60 dias sem encerramento sinalizam redistribuição ou escalonamento.

```
SELECT d.id, d.titulo, d.tipo_demanda, d.status,

d.prioridade, n.nome AS nucleo, p.nome AS responsavel,

d.prazo_interno,

CURRENT_DATE - d.criado_em::date AS dias_aberto,

CURRENT_DATE - MAX(m.ocorrido_em)::date AS dias_sem_mov,

CASE

WHEN CURRENT_DATE - d.criado_em::date > 60 THEN 'critico'

WHEN CURRENT_DATE - d.criado_em::date > 30 THEN 'atencao'

WHEN CURRENT_DATE - d.criado_em::date > 15 THEN 'monitorar'

ELSE 'normal'

END AS faixa_aging

FROM demandas d

JOIN nucleos n ON n.id = d.nucleo_id

JOIN alocaoes a ON a.demanda_id = d.id

AND a.responsavel_principal = TRUE

JOIN profissionais p ON p.id = a.profissional_id

LEFT JOIN movimentacoes m ON m.demanda_id = d.id

WHERE d.status NOT IN ('encerrado', 'arquivado')

GROUP BY d.id, d.titulo, d.tipo_demanda, d.status,

d.prioridade, n.nome, p.nome, d.prazo_interno, d.criado_em

ORDER BY dias_aberto DESC;
```

KPI 4 — Throughput Mensal

Demandas encerradas por mês, núcleo e profissional. Calcula tempo médio de encerramento (TME em dias) e throughput por profissional ativo. Base para forecast de capacidade.

```
SELECT TO_CHAR(a.encerrado_em, 'YYYY-MM') AS mes,  
  
n.nome AS nucleo,  
  
COUNT(DISTINCT d.id) AS demandas_encerradas,  
  
AVG(EXTRACT(EPOCH FROM  
  
a.encerrado_em - d.criado_em) / 86400)::int  
  
AS tme_dias,  
  
SUM(a.horas_realizadas) AS horas_totais,  
  
ROUND(AVG(a.horas_realizadas),1) AS horas_media,  
  
COUNT(DISTINCT a.profissional_id) AS profissionais_ativos,  
  
ROUND(COUNT(DISTINCT d.id)::numeric /  
  
NULLIF(COUNT(DISTINCT a.profissional_id),0),1)  
  
AS throughput_por_prof  
  
FROM alocoes a  
  
JOIN demandas d ON d.id = a.demanda_id  
  
JOIN nucleos n ON n.id = d.nucleo_id  
  
WHERE a.responsavel_principal = TRUE  
  
AND a.encerrado_em >= CURRENT_DATE - INTERVAL '12 months'  
  
AND d.status = 'encerrado'  
  
GROUP BY mes, n.nome  
  
ORDER BY mes DESC, n.nome;
```

KPI 5 — Carteira Financeira

Valor total das causas ativas por profissional, núcleo e tipo de demanda. Distribui por faixas de valor (alto > R\$ 500k / médio / baixo). Conecta operação ao EBITDA do escritório.

```
SELECT n.nome AS nucleo, p.nome AS profissional,

d.tipo_demanda, COUNT(*) AS total_causas,

SUM(d.valor_causa)

FILTER (WHERE d.status NOT IN ('encerrado','arquivado'))

AS valor_carteira_ativa,

AVG(d.valor_causa)

FILTER (WHERE d.status NOT IN ('encerrado','arquivado'))

AS valor_medio_causa,

COUNT(*) FILTER (WHERE d.valor_causa > 500000)

AS causas_alto_valor,

COUNT(*) FILTER (WHERE d.valor_causa BETWEEN 50000 AND 500000)

AS causas_medio_valor,

COUNT(*) FILTER (WHERE d.valor_causa < 50000)

AS causas_baixo_valor

FROM demandas d

JOIN nucleos n ON n.id = d.nucleo_id

JOIN alocacoes a ON a.demanda_id = d.id

AND a.responsavel_principal = TRUE

JOIN profissionais p ON p.id = a.profissional_id

WHERE d.valor_causa IS NOT NULL

GROUP BY n.nome, p.nome, d.tipo_demanda

ORDER BY valor_carteira_ativa DESC NULLS LAST;
```


KPI 6 — View Materializada · Painel do Gestor

Visão consolidada por núcleo consumida pelo dashboard de gestão em uma única chamada. Refresh automático a cada 15 minutos via pg_cron. Elimina N+1 queries no frontend.

```
CREATE MATERIALIZED VIEW mv_painel_gestor AS

SELECT n.id AS nucleo_id, n.nome AS nucleo,

COUNT(DISTINCT p.id) AS total_profissionais,

COUNT(DISTINCT d.id)

FILTER (WHERE d.status = 'execucao')

AS em_execucao,

COUNT(DISTINCT d.id)

FILTER (WHERE d.prazo_interno < CURRENT_DATE

AND d.status NOT IN ('encerrado','arquivado'))

AS vencidas,

COUNT(DISTINCT d.id)

FILTER (WHERE d.prazo_interno

BETWEEN CURRENT_DATE AND CURRENT_DATE + 2)

AS vencendo_48h,

COUNT(DISTINCT d.id)

FILTER (WHERE d.status = 'triagem')

AS aguardando_triagem,

ROUND(AVG(p.carga_atual::numeric /

NULLIF(p.carga_maxima,0) * 100),1)

AS pct_ocupacao_media,

MAX(d.atualizado_em) AS ultima_movimentacao

FROM nucleos n
```

```
LEFT JOIN profissionais p ON p.nucleo_id = n.id AND p.ativo
```

```
LEFT JOIN demandas d ON d.nucleo_id = n.id
```

```
GROUP BY n.id, n.nome;
```

```
CREATE UNIQUE INDEX ON mv_painel_gestor (nucleo_id);
```

```
-- Agendar refresh via pg_cron:
```

```
SELECT cron.schedule('refresh-painel',
```

```
'*/15 * * * *',
```

```
'REFRESH MATERIALIZED VIEW CONCURRENTLY mv_painel_gestor;');
```

5. Roadmap de Implementação

Cinco sprints de duas semanas cada (10 semanas no total). O MVP ao final do Sprint 3 já entrega valor para a controladoria com demandas, alocações e alertas de SLA funcionando.

Sprint 1	Semanas 1–2	Fundação
		• Configurar banco PostgreSQL + schema completo
		• Criar projeto FastAPI com autenticação JWT
		• Implementar CRUD de núcleos e profissionais
		• Setup inicial do frontend React (rotas e autenticação)
		• Configurar Redis para sessão e filas

Sprint 2	Semanas 3–4	Core de Demandas
		• CRUD completo de demandas com validações
		• Motor de alocação com verificação de carga máxima
		• Workflow engine: transições de status com regras
		• Registro automático de movimentações (audit trail)
		• Upload de documentos para Object Storage

Sprint 3	Semanas 5–6	SLA e Notificações (MVP)
		• SLA Engine: cálculo de prazos em dias úteis
		• Job assíncrono (Celery/Redis) para varredura de gatilhos
		• Integração WhatsApp API para alertas de prazo
		• Dashboard básico: demandas por status e prazo
		• ENTREGA MVP — escritório já pode usar o sistema

Sprint 4	Semanas 7–8	Controladoria e BI
		• Implementar as 6 queries de KPI como endpoints REST
		• View materializada mv_painel_gestor com pg_cron
		• Dashboard de gestão com KPIs em tempo real
		• Conector Power BI via DirectQuery
		• Relatórios exportáveis (CSV / Excel)

Sprint 5	Semanas 9–10	Integrações e Polimento
		• Integração TJRS: importar andamentos e prazos
		• Integração Google Drive: vincular peças às demandas
		• PWA mobile: notificações push
		• Testes de carga e ajuste de índices
		• Documentação de API (Swagger) e manual do usuário

6. Stack Tecnológica e Dependências

	Tecnologia	Versão mín.	Justificativa
API	Python + FastAPI	3.11 / 0.110	Performance assíncrona, tipagem Pydantic, Swagger automático
Banco de dados	PostgreSQL	15+	JSONB, CTEs, window functions, pg_cron para jobs
Filas	Redis	7+	Sessão JWT, cache de views, filas Celery para alertas
Tarefas async	Celery	5+	Jobs de SLA, envio de notificações, refresh de views
Frontend	React + TypeScript	18 / 5+	SPA com roteamento protegido por perfil RBAC
	PWA (Vite + Workbox)	—	Notificações push sem app nativo, deploy único
Autenticação	JWT + OAuth2	—	Tokens stateless, refresh token com Redis allowlist
Armazenamento	S3-compatible (MinIO)	—	Documentos e peças, path por demanda/profissional
Analytics	Power BI DirectQuery	—	Consome PostgreSQL direto, sem ETL adicional
	Docker + Railway/VPS	—	Containerizado, CI/CD via GitHub Actions
Monitoramento	Sentry + pg_stat	—	Erros de runtime e slow queries em produção

requirements.txt — Backend

```
fastapi==0.110.0

uvicorn[standard]==0.27.0

pydantic==2.6.0

sqlalchemy==2.0.27

asyncpg==0.29.0 # driver async PostgreSQL

alembic==1.13.1 # migrations

redis==5.0.1

celery==5.3.6

python-jose[cryptography]==3.3.0 # JWT

passlib[bcrypt]==1.7.4

boto3==1.34.0 # S3 / MinIO
```

```
httpx==0.26.0 # cliente HTTP async
```

```
sentry-sdk==1.40.0
```

7. Variáveis de Ambiente — .env

```
# Banco de dados
```

```
DATABASE_URL=postgresql+asyncpg://user:pass@host:5432/demandas
```

```
# Redis
```

```
REDIS_URL=redis://localhost:6379/0
```

```
# JWT
```

```
SECRET_KEY=
```

```
ACCESS_TOKEN_EXPIRE_MINUTES=480 # 8 horas
```

```
REFRESH_TOKEN_EXPIRE_DAYS=30
```

```
# Object Storage (MinIO ou S3)
```

```
S3_ENDPOINT=https://seu-bucket.s3.amazonaws.com
```

```
S3_ACCESS_KEY=...
```

```
S3_SECRET_KEY=...
```

```
S3_BUCKET=demandas-docs
```

```
# WhatsApp API (Meta Business)
```

```
WA_TOKEN=...
```

```
WA_PHONE_NUMBER_ID=...
```

```
# Sentry
```

```
SENTRY_DSN=https://...

# Ambiente

ENV=production # development | staging | production
```

8. Checklist de Partida — Sprint 1

Infraestrutura

[] Provisionar servidor (VPS 4 vCPU / 8 GB RAM mínimo)
[] Instalar Docker e Docker Compose
[] Configurar PostgreSQL 15 com database 'demandas'
[] Subir Redis 7 em container
[] Configurar bucket S3/MinIO para documentos
[] Apontar domínio e configurar SSL (Let's Encrypt)

Código — Backend

[] Criar repositório GitHub e branch strategy (main / develop / feature/*)
[] Inicializar projeto FastAPI com estrutura de módulos
[] Configurar Alembic para migrations
[] Executar migration do schema SQL (seção 2)
[] Implementar autenticação JWT com refresh token
[] Criar endpoints CRUD de nucleos e profissionais
[] Adicionar middleware RBAC (seção 3)

Código — Frontend

[] Inicializar projeto React + TypeScript + Vite
[] Configurar React Router com rotas protegidas por perfil
[] Criar tela de login com validação JWT
[] Criar CRUD de demandas (formulário + listagem + detalhe)
[] Configurar interceptor Axios para refresh automático do token

Validação

☐ Testar criação de demanda do portal ao banco
☐ Validar restrições RBAC: advogado nao vê demandas de outros
☐ Confirmar que atualizado_em é atualizado automaticamente
☐ Testar upload e download de documento vinculado
☐ Verificar que carga_atual incrementa ao alocar profissional

Nota sobre este documento

Este documento foi gerado pela Crescer Tecnologia e Gestão como especificação técnica inicial para o Sistema de Controle de Demandas. Todos os schemas SQL, queries e configurações foram projetados para PostgreSQL 15+ e Python 3.11+. Adapte os parâmetros de carga máxima, SLA e perfis de acesso conforme a estrutura específica do escritório antes da implementação.

Para dúvidas sobre a implementação ou para contratar o desenvolvimento completo do sistema, entre em contato: crescertecnologia.com · @arquitetojuridico.ia